VR 人形机器人遥操作项目阶段路线

本文档按照项目发展的时间顺序，记录从原始 HTS 手部追踪测试，到脚部和全身关节扩展，再到数据录制、离线回放、机器人模拟器接入和 VR 视觉回传的完整路线。文档的写法采用“一个阶段接一个阶段”的形式，不单独区分已完成和未完成部分，方便作为项目整体规划和开发记录使用。

项目最终目标：

```
机器人视觉/状态 -> VR 设备显示
VR 用户头部、手部、身体动作 -> 控制机器人
```

也就是构建一个面向人形机器人的 VR 遥操作系统。

1. 总体系统结构

系统由两条数据流组成。

机器人到 VR：

```
机器人相机 / 深度传感器 / 状态信息
        ↓
PC 或机器人端编码与转发
        ↓
Unity / Quest 接收
        ↓
VR 中显示机器人第一视角、状态 HUD 或深度信息
```

VR 到机器人：

```
Quest 获取用户头部、手部、身体动作
        ↓
Unity BodyLandmarkStreamer 输出全身关节
        ↓
Python 中间层接收、记录、可视化
        ↓
坐标转换、尺度归一化、滤波、IK、安全限制
        ↓
机器人模拟器
        ↓
真实机器人
```

系统中每一层的职责需要保持清晰：

- Unity/Quest 层负责采集 VR 设备可获得的人体数据。
- Python 接收层负责把 Unity 发来的文本数据解析成结构化数组。
- Python 可视化和录制层负责检查数据是否稳定、连续、方向正确。
- 机器人映射层负责从人体骨架中提取机器人控制目标。
- 机器人控制层负责 IK、滤波、限位、速度约束和安全保护。
- VR 显示层负责把机器人视觉和状态回传给用户。

整体开发原则：

- 先验证数据，再控制机器人。
- 先离线回放，再实时控制。
- 先控制头部和双手，再扩展到全身。
- 先在模拟器中验证，再接真实机器人。
- 每一步都保留可视化和日志，避免黑箱调试。

2. 阶段 0：原始 HTS 手部追踪验证

目标：确认原始 hand-tracking-streamer 项目可以运行，并建立最基础的数据链路。

这个阶段的核心是不要立刻修改功能，而是先确认原始项目的手部数据从 Quest 到 Python 的链路是通的。只有原始手部链路稳定，后续扩展脚部和全身才有可靠基础。

工作内容：

- 打开原始 Unity HTS 项目。
- 确认 Unity 项目可以正常编译。
- 在 Quest 中启用手部追踪权限。
- 运行 Unity 场景或构建到 Quest。
- 确认 Unity 中的 hand streamer 能输出左右手 wrist 和 landmarks。
- 在 PC 端运行 socket 接收脚本。
- 通过 TCP 或 UDP 接收 Unity 发来的文本消息。
- 使用原始 `visualizer.py` 显示左右手关节。

相关脚本：

```
scripts/sockets.py
scripts/visualizer.py
```

典型 TCP 调试命令：

```
adb reverse tcp:8000 tcp:8000
python scripts/sockets.py --protocol tcp --host localhost --port 8000
```

典型 UDP 调试命令：

```
python scripts/sockets.py --protocol udp --host 0.0.0.0 --port 9000
```

典型可视化命令：

```
python scripts/visualizer.py --protocol tcp --host localhost --port 8000 --show-fingers
```

需要确认的数据内容：

```
Left wrist
Right wrist
Left landmarks
Right landmarks
```

需要观察的现象：

- 左手移动时左手点云移动。
- 右手移动时右手点云移动。
- 手腕位置和手指 landmarks 大致连续。
- TCP/UDP 消息没有明显中断。
- Python 可视化窗口不会频繁卡死。

常见问题：

- Unity 编译失败时，先看 Console 中的 C# compile error。
- TCP 没有数据时，先确认 `adb reverse tcp:8000 tcp:8000` 是否执行。
- UDP 没有数据时，检查 Quest 和 PC 是否在同一网络，以及防火墙是否阻止端口。
- Python 缺依赖时，安装 `numpy` 和 `matplotlib`。

检查点：

```
Quest / Unity -> TCP/UDP -> Python -> 手部可视化
```

3. 阶段 1：脚部和腿部关节扩展

目标：在原始手部追踪基础上，尝试获取脚部和腿部相关关节。

这个阶段的关键认识是：脚部数据不是 Hand Tracking 的扩展，而是 Meta Body Tracking 的一部分。因此 Unity 中不能只寻找“脚部 tracking”选项，而需要启用 Body Tracking，并使用 `Body` 组件提供的 FullBody skeleton。

工作内容：

- 新增 `FootLandmarkStreamer.cs`。
- 在 Unity 中启用 Meta Body Tracking。
- 在 Oculus/Meta runtime settings 中启用 Full Body joint set。
- 确认应用启动时请求 Body Tracking 权限。
- 将 foot streamer 挂载到带有 `Body` 组件的 GameObject。
- 避免把 streamer 同时挂在 `OVRBody` 和 `OVRBodyDataSource` 两处。
- 通过 Unity 日志确认 Body Tracking 状态。
- 输出脚踝、膝盖、脚跟、脚尖等脚部/腿部相关关节。
- 在 Python 端扩展脚部数据解析和可视化逻辑。

Unity 层级检查原则：

```
streamer 应挂载在有 Body 组件的对象上。
数据源对象只负责提供数据，不应重复挂同一个 streamer。
```

Body Tracking 需要检查的状态：

```
Connected
Valid
HighConf
UpdatedEvent
FullBodyFoot
```

典型日志目标：

```
Connected=True
Valid=True
HighConf=True
UpdatedEvent=True
FullBodyFoot=True
```

脚部阶段常用的关节目标：

```
LeftKnee
RightKnee
LeftAnkle
RightAnkle
LeftFootHeel
RightFootHeel
LeftFootBall
RightFootBall
```

Python 可视化需要观察：

- 左脚和右脚没有交换。
- 膝盖、脚踝、脚尖的顺序合理。
- 抬左腿时左侧腿部数据变化。
- 抬右腿时右侧腿部数据变化。
- 脚部点不是固定不动。
- 数据更新频率和手部类似。

关键结论：

```
脚部不是独立的 Hand Tracking 功能，而是来自 Meta Body Tracking 的 FullBody skeleton。
```

常见问题：

- `Connected=True` 但 `UpdatedEvent=False` 时，通常需要检查 `OVRBodyDataSource` 和 `Body` 的连接关系。
- 同一对象层级中挂了多个 streamer 时，日志可能出现重复或左右混乱。
- Quest 没有给 Body Tracking 权限时，Unity 端可能能运行但没有有效数据。
- 脚部数据本身是推断结果，稳定性通常弱于手部。

检查点：

```
Quest / Unity -> FootLandmarkStreamer -> Python -> 脚部和腿部可视化
```

4. 阶段 2：全身 84 个关节输出

目标：从脚部扩展到完整全身关节输出。

这个阶段不再只针对脚部，而是把 Meta FullBody skeleton 中的所有关节统一输出。这样后续机器人映射时可以自由选择头、肩、肘、手腕、骨盆、膝盖、脚踝等关键点，而不是被脚部 streamer 的子集限制。

工作内容：

- 新增 `BodyLandmarkStreamer.cs`。
- 从 Meta FullBody skeleton 获取完整关节数据。
- 输出 `Body root`。
- 输出 `Body joints_pose`。
- 每个关节包含世界位置和世界旋转。
- 处理 Body 更新事件，避免递归刷新导致程序崩溃。
- 通过 Unity 日志确认每帧有效关节数量。
- 在 `CONNECTIONS.md` 中记录全身数据协议。

全身数据格式：

```
Body root:, x, y, z, qx, qy, qz, qw
Body joints_pose:, [x, y, z, qx, qy, qz, qw] * 84
```

每个关节 pose 的含义：

```
x, y, z       世界位置
qx, qy, qz    四元数向量部分
qw            四元数标量部分
```

Unity 日志中需要关注：

```
Streaming full body
Valid joints: 84/84
High confidence: True
```

设计注意点：

- `BodyLandmarkStreamer` 应该作为全身数据的统一出口。
- 测试全身数据时，可以暂时移除或禁用 `FootLandmarkStreamer`。
- 不要在多个 GameObject 上重复挂多个全身 streamer，否则 Python 端会收到重复帧。
- 如果只需要全身数据，后续机器人控制不应依赖旧的手部/脚部单独协议。

常见问题：

- 程序运行一会儿闪退时，检查是否在 `WhenBodyUpdated` 回调中又触发了 Body 数据刷新。
- 如果 Unity 日志显示 84/84 但 Python 没有更新，检查网络端口和协议是否一致。
- 如果左右侧混乱，先检查 Unity 端只保留一个全身 streamer。

检查点：

```
Quest / Unity -> BodyLandmarkStreamer -> Body root + Body joints_pose
```

5. 阶段 3：Python 实时全身可视化

目标：在 Python 中实时检查全身数据的位置、方向和连续性。

这个阶段的作用是把 Unity 发出的 84 个关节从抽象数字变成可观察的骨架。可视化不是最终目标，但它是后续机器人映射前最重要的调试工具。

工作内容：

- 扩展 `scripts/visualizer.py`。
- 增加对以下消息的解析：

```
Body root
Body joints
Body joints_pose
```

- 将 Unity 坐标转换到 Python 可视化坐标。
- 绘制完整身体骨架连线。
- 隐藏容易造成误解的 root 点，避免视觉上出现“三条腿”。
- 增加头部可视化：
  - 头部关节点小红点。
  - 头部方向轴。
  - 头部立方体线框。
- 调整头部立方体的位置和尺寸，使其更容易和颈椎区分。

坐标转换目的：

```
Unity/Quest 坐标需要转换成 Python 可视化中更直观的右手坐标。
否则前后、左右或高度方向可能看起来不符合直觉。
```

头部可视化设计：

- 小红点表示真实头部关节点。
- 立方体表示头部大致体块。
- 方向轴表示头部旋转。
- 立方体略高于头部关节点，便于和颈椎区分。

典型可视化命令：

```
python scripts/visualizer.py --protocol tcp --host localhost --port 8000 --show-fingers
```

需要做的检查动作：

- 原地站立几秒。
- 左右摇头。
- 点头。
- 抬左手。
- 抬右手。
- 抬左腿。
- 抬右腿。
- 身体轻微转向。

观察重点：

- 头部方向轴是否随摇头变化。
- 左右手是否没有镜像反转。
- 左右脚是否没有交换。
- 肩、肘、腕是否连线合理。
- 髋、膝、踝是否连线合理。
- 身体没有明显跳变。

常见问题：

- 出现“三条腿”通常是 root 到骨盆的连线造成视觉误解，可以隐藏 root 或移除 root 连线。
- 头部不明显时，可以增加方向轴、头部方块或颜色标记。
- 可视化抖动不一定意味着网络错误，也可能是 Body Tracking 推断本身不稳定。

检查点：

```
Python 实时显示全身骨架，头部方向随摇头变化。
```

6. 阶段 4：全身数据录制

目标：将实时全身关节数据保存下来，作为后续机器人映射和模拟器调试的数据源。

这个阶段的核心是获得可重复使用的数据。实时数据每次都不一样，而录制数据可以反复回放、反复调试坐标转换、IK 和机器人控制逻辑。

工作内容：

- 新增录制脚本：

```
scripts/record_body_stream.py
```

- 支持 TCP/UDP 接收。
- 支持指定录制时长。
- 支持 Ctrl+C 停止后保存。
- 保存为 `.npz` 文件。
- 保存时间戳、root pose 和所有关节 pose。
- 在没有收到完整帧时避免写出空文件。

输出数据格式：

```
timestamps       [N]
root_poses       [N, 7]
joint_poses      [N, 84, 7]
joint_positions  optional
joint_count
metadata
```

字段说明：

```
timestamps       每帧相对录制开始的时间
root_poses       每帧 root 的 position + quaternion
joint_poses      每帧 84 个关节的 position + quaternion
joint_positions  如果收到无旋转版本 joints，则可额外保存
metadata         协议、端口、关节数量等录制信息
```

典型命令：

```
adb reverse tcp:8000 tcp:8000
python scripts/record_body_stream.py --protocol tcp --host localhost --port 8000 --duration 10
```

指定输出文件：

```
python scripts/record_body_stream.py --protocol tcp --host localhost --port 8000 --duration 10 -o recordings/stand_01.npz
```

建议录制的数据集：

```
stand_01.npz       原地站立
head_hand_01.npz   转头和抬手
leg_01.npz         抬腿和屈膝
turn_01.npz        身体轻微转向
mixed_01.npz       头、手、腿组合动作
```

录制时注意：

- 每段动作开始前先静止 2-3 秒，方便后续标定初始姿态。
- 动作不要一开始就太快，先保证数据连续。
- 每段动作最好控制在 10-20 秒，便于调试。
- 文件名应反映动作内容。

检查点：

```
Quest / Unity -> Python recorder -> recordings/*.npz
```

7. 阶段 5：记录与可视化同步进行

目标：录制数据时同时看到实时骨架，避免录完后才发现动作或坐标有问题。

这个阶段把实时检查和数据录制合并到同一个脚本中。这样录制期间可以立即发现 tracking 丢失、左右反转、头部方向异常或身体跳变。

工作内容：

- 新增脚本：

```
scripts/record_visualize_body_stream.py
```

- 同时执行：

```
实时接收 Body joints_pose
实时显示全身骨架
实时累计录制帧
退出时保存 .npz
```

典型命令：

```
adb reverse tcp:8000 tcp:8000
python scripts/record_visualize_body_stream.py --protocol tcp --host localhost --port 8000 --duration 10
```

指定输出文件：

```
python scripts/record_visualize_body_stream.py --protocol tcp --host localhost --port 8000 --duration 10 -o recordings/head_
```

录制窗口中需要观察：

- 标题栏中的 recorded frames 是否增加。
- 骨架是否持续刷新。
- 头部立方体是否随头部移动。
- 方向轴是否随头部转动。
- 录制期间是否出现明显断帧。

适合使用该脚本的情况：

- 第一次录某类动作。
- 调整 Unity 端 streamer 之后。
- 改动 Python 坐标转换之后。
- 需要快速确认文件中确实有完整 body frames。

检查点：

```
边看边录，得到可回放的 recordings/*.npz。
```

8. 阶段 6：录制数据离线回放

目标：对 `.npz` 录制文件进行离线检查，并能反复查看同一段动作。

离线回放是进入机器人控制前的重要步骤。机器人控制中的很多问题，例如左右反转、坐标轴错误、尺度不合适，最好先在离线人体骨架中确认，而不是接到模拟器后再排查。

工作内容：

- 新增脚本：

```
scripts/replay_body_recording.py
```

- 支持普通速度播放。
- 支持慢速播放。
- 支持循环播放。
- 支持跳帧快速检查。
- 使用与实时可视化一致的身体骨架、头部小点、头部立方体和方向轴。

典型命令：

```
python scripts/replay_body_recording.py recordings/test_01.npz --loop
```

慢速检查：

```
python scripts/replay_body_recording.py recordings/test_01.npz --speed 0.3 --loop
```

快速检查：

```
python scripts/replay_body_recording.py recordings/test_01.npz --frame-step 3
```

检查内容：

- 左右手是否正确。
- 左右脚是否正确。
- 头部方向轴是否随摇头变化。
- 身体骨架是否连续。
- 抬手、抬腿、转头动作是否正确回放。
- 静止站立时骨盆和脚部是否稳定。
- 四元数方向是否明显异常。

建议记录检查结果：

```
文件名
动作内容
是否左右正确
是否头部方向正确
是否有跳变
是否适合用于机器人映射
```

常见问题：

- 回放窗口不动时，检查 `.npz` 中是否有 `joint_poses`。
- 动作很快时，使用 `--speed 0.3`。
- 动作太长时，使用 `--frame-step` 快速扫一遍。
- 头部方向轴不对时，优先检查四元数坐标转换。

检查点：

```
recordings/*.npz -> replay_body_recording.py -> 离线全身骨架回放
```

9. 阶段 7：提取机器人控制关键点

目标：从 84 个全身关节中提取机器人控制真正需要的少量关键点。

机器人通常不能直接使用人体的 84 个关节点。人和机器人的骨架结构、自由度、比例和关节定义都不同。更合理的方法是先把人体骨架压缩成机器人可用的控制目标，例如头部方向、左右手末端位置、骨盆位置、左右脚位置等。

建议提取：

```
Pelvis / Root
Head
LeftShoulder
RightShoulder
LeftElbow
RightElbow
LeftWrist
RightWrist
LeftHip
RightHip
LeftKnee
RightKnee
LeftAnkle
RightAnkle
```

建议脚本：

```
scripts/extract_robot_targets.py
```

输入：

```
recordings/*.npz
```

输出：

```
robot_targets/*.npz
```

建议输出内容：

```
timestamps
pelvis_position
pelvis_quat
head_position
head_quat
left_shoulder_position
right_shoulder_position
left_elbow_position
right_elbow_position
left_wrist_position
right_wrist_position
left_hip_position
right_hip_position
left_knee_position
right_knee_position
left_ankle_position
right_ankle_position
```

提取逻辑需要包含：

- 从 `joint_poses[:, joint_index, :]` 中取出指定关节。
- 保留原始时间戳。
- 保留必要的 position 和 quaternion。
- 输出一个更小、更适合机器人控制的数据文件。
- 可选地输出 JSON metadata，记录每个 key 对应的 Meta joint index。

建议增加一个关键点可视化脚本：

```
scripts/visualize_robot_targets.py
```

该脚本只显示机器人控制目标点：

```
head
pelvis
left wrist
right wrist
left elbow
right elbow
left ankle
right ankle
left knee
right knee
```

检查内容：

- 提取出来的关键点和原始骨架位置一致。
- 左右关键点没有交换。
- 头部四元数保留正确。
- 时间戳数量和原始录制一致。
- 输出文件比原始 84 关节文件更简单。

检查点：

```
recordings/*.npz -> extract_robot_targets.py -> robot_targets/*.npz
```

10. 阶段 8：人体到机器人坐标转换

目标：把 Quest/Unity 的人体坐标转换到机器人模拟器坐标。

坐标转换是后续机器人控制中最容易出错的部分。即使人体骨架看起来正确，只要机器人模拟器坐标系和人体坐标系不一致，就会出现抬左手变成抬右手、向前变成向侧方、头部朝向错误等问题。

需要处理：

- 坐标轴方向。
- 左右是否反转。
- 高度方向。
- 初始朝向。
- 人体尺度和机器人尺度差异。
- 初始站姿对齐。

Unity/Quest 常用坐标：

```
x: 右
y: 上
z: 前
```

许多机器人模拟器常用坐标：

```
x: 前
y: 左
z: 上
```

建议脚本：

```
scripts/calibrate_body_to_robot.py
```

输入：

```
静止站立 recording
```

输出：

```
calibration/body_to_robot_calibration.json
```

标定文件建议包含：

```
axis_mapping
scale
origin_offset
initial_pelvis_position
initial_head_forward
user_height_estimate
robot_height
```

标定步骤：

- 读取一段静止站立数据。
- 取前几秒的 pelvis、head、left ankle、right ankle 平均值。
- 用脚踝或骨盆估计地面高度。
- 用 head 到 foot 的距离估计用户身高。
- 计算人体尺度到机器人尺度的缩放比例。
- 计算人体初始朝向到机器人初始朝向的旋转。
- 保存坐标转换参数。

验证动作：

- 静止站立。
- 抬左手。
- 抬右手。
- 向前伸手。
- 左右转头。

检查内容：

- 抬左手后机器人目标中的 left wrist 上升。
- 抬右手后机器人目标中的 right wrist 上升。
- 向前伸手后目标点沿机器人前向移动。
- 头部前向和机器人前向一致。
- 手脚目标不会因为尺度问题离机器人过远。

检查点：

```
人体关键点 -> 坐标转换 -> 机器人坐标系关键点
```

11. 阶段 9：接入人形机器人模拟器

目标：选择并加载一个人形机器人模型，为后续 IK 和控制做准备。

建议优先使用：

```
MuJoCo + Python
```

原因：

- 适合人形机器人控制。
- Python 接口方便。
- 便于离线回放、实时控制和 learning-based control。
- 可以方便地访问关节位置、速度、执行器和传感器。

需要准备：

- 一个可运行的人形机器人 MJCF/XML 模型。
- 能在 MuJoCo 中加载并显示模型。
- 能读取机器人关节状态。
- 能设置机器人关节目标或 actuator control。
- 能以固定 timestep 运行仿真。

建议脚本：

```
scripts/mujoco_load_robot.py
```

脚本需要做的事：

- 加载 MJCF/XML 模型。
- 打开 MuJoCo viewer。
- 打印机器人 joint names。
- 打印 actuator names。
- 打印 body/site names。
- 设置一个初始站姿。
- 让仿真稳定运行。

需要整理的信息：

```
机器人头部关节名称
机器人左肩/右肩关节名称
机器人左肘/右肘关节名称
机器人左腕/右腕或末端 site 名称
机器人骨盆/root body 名称
机器人左右脚 site 名称
关节角限制
执行器控制范围
```

检查内容：

- 机器人模型能正常显示。
- 初始姿态不会立刻倒下或爆炸。
- 手动设置关节角能看到对应身体部位运动。
- 机器人左右关节命名清楚。
- 找到手腕、头部、脚部的 site 或 body。

检查点：

```
MuJoCo 打开机器人模型，并能手动改变关节角。
```

12. 阶段 10：离线人体动作驱动机器人

目标：先使用录制数据驱动机器人模拟器，而不是直接实时控制。

这个阶段使用固定的 `.npz` 录制数据驱动机器人。因为输入数据固定，所以如果机器人动作不对，问题通常来自坐标转换、尺度、IK 或关节限制，而不是实时网络波动。

第一步控制上半身：

```
VR head        -> 机器人头部方向
VR left wrist  -> 机器人左手末端目标
VR right wrist -> 机器人右手末端目标
```

第二步加入更多身体目标：

```
left elbow / right elbow
pelvis
left ankle / right ankle
left knee / right knee
```

建议脚本：

```
scripts/mujoco_replay_body_targets.py
```

控制方法：

- 使用 IK 求机器人关节角。
- 不直接把人体关节点位置当作机器人关节角。
- 先控制头部和双手，再扩展到全身。
- 加入关节角限制和速度限制。
- 对目标点做低通滤波。
- 对不可达目标做裁剪。

IK 目标建议：

```
head orientation
left hand position
right hand position
left elbow optional guide
right elbow optional guide
pelvis position optional
```

离线回放流程：

```
recordings/*.npz
        ↓
extract_robot_targets.py
        ↓
calibrate_body_to_robot.py
        ↓
mujoco_replay_body_targets.py
        ↓
MuJoCo robot motion
```

调试顺序：

- 只让机器人头部跟随。
- 再让左手跟随。
- 再让右手跟随。
- 再同时控制头部和双手。
- 再加入肘部 guide。
- 最后考虑腿部或骨盆。

检查内容：

- 头部方向是否正确。
- 左右手是否没有交换。
- 手部目标是否在机器人可达范围内。
- IK 是否平滑。
- 关节角是否超限。
- 机器人是否出现突然抖动或穿模。

检查点：

```
robot_targets/*.npz -> MuJoCo IK -> 机器人头部和双手跟随录制动作
```

13. 阶段 11：实时 VR 到机器人模拟器控制

目标：把离线流程改成实时控制。

这个阶段把固定 `.npz` 文件替换成 Unity/Quest 实时输入。离线版本中已经验证过的坐标转换、尺度标定、目标提取和 IK 逻辑应尽量复用，避免重新写一套。

实时链路：

```
Quest / Unity
        ↓
TCP / UDP
        ↓
Python receiver
        ↓
坐标转换 + 滤波 + IK + 安全限制
        ↓
MuJoCo
```

建议脚本：

```
scripts/mujoco_live_body_bridge.py
```

控制层需要包含：

- 低通滤波。
- 最大速度限制。
- 最大关节角限制。
- 网络断开保护。
- 目标丢失保护。
- 急停按键。
- 实时日志。
- 控制频率统计。

实时控制需要关注的频率：

```
Unity 发送频率
Python 接收频率
IK 求解频率
MuJoCo 仿真频率
控制命令更新频率
```

安全策略：

- 如果超过一定时间没有收到新 body frame，保持当前机器人姿态或回到安全姿态。
- 如果目标点突然跳变，限制最大移动速度。
- 如果 IK 解超出关节范围，裁剪到安全范围。
- 如果用户按下急停，立即停止发送控制命令。

调试顺序：

- 先实时控制头部。
- 再实时控制单手。
- 再实时控制双手。
- 再加入头部和双手同时控制。
- 再加入更复杂的躯干或腿部控制。

检查点：

```
Quest 实时动作 -> MuJoCo 机器人头部和双手实时跟随
```

14. 阶段 12：机器人视觉回传到 VR

目标：把机器人相机画面显示到 Quest 中。

视觉回传是闭环遥操作的另一半。用户不仅要控制机器人，还需要通过 VR 设备看到机器人观察到的环境。

最小版本：

```
机器人相机
        ↓
PC 接收视频流
        ↓
Unity Texture
        ↓
Quest 中显示在一个前方平面上
```

扩展版本：

```
双目相机 -> VR 双目显示
深度图   -> 点云/深度辅助显示
状态数据 -> VR HUD
```

建议顺序：

```
1. 单目 RGB 视频回传。
2. Unity 中显示为纹理。
3. 加入延迟测量。
4. 扩展双目或深度显示。
```

可能的视频来源：

- 真实机器人 RGB 相机。
- MuJoCo/Isaac/Gazebo 模拟器相机。
- ROS image topic。
- RTSP stream。
- WebRTC stream。
- 自定义 TCP/UDP 图像流。

Unity 端显示方式：

- 将视频帧更新到 Texture2D。
- 将纹理显示在 VR 前方固定平面。
- 后续可以改为曲面屏、双目显示或 passthrough overlay。
- 在画面旁边显示机器人状态 HUD。

需要测量：

```
视频帧率
端到端延迟
丢帧率
分辨率
Unity 解码开销
Quest 显示稳定性
```

检查点：

```
机器人相机画面 -> Unity Texture -> Quest VR 显示
```

15. 阶段 13：闭环 VR 遥操作

目标：用户在 VR 中看到机器人视角，同时用身体动作控制机器人。

闭环结构：

```
用户在 Quest 中看到机器人视角
        ↓
用户移动头部 / 手部 / 身体
        ↓
Python 控制层生成机器人动作
        ↓
机器人执行
        ↓
机器人相机画面回到 Quest
```

系统需要包含：

- 急停。
- 工作空间限制。
- 碰撞检测。
- 关节限位。
- 速度限制。
- 延迟监控。
- 数据丢失保护。
- 操作模式切换。
- 机器人状态显示。

建议操作模式：

```
观察模式：只显示机器人视觉，不发送动作控制。
头部控制模式：只控制机器人头部或相机云台。
上肢控制模式：控制头部和双手。
全身控制模式：加入躯干、腿部或移动底盘。
安全停止模式：立即停止控制输出。
```

基础任务设计：

- 用户左右转头，机器人视角跟随。
- 用户抬起左手，机器人左手抬起。
- 用户抬起右手，机器人右手抬起。
- 用户伸手靠近目标，机器人末端靠近目标。
- 用户在 VR 中根据机器人相机画面完成简单指向或抓取前置动作。

闭环系统需要记录：

```
VR body frame timestamp
robot command timestamp
robot state timestamp
camera frame timestamp
end-to-end latency
control frequency
video frequency
```

检查点：

```
VR 视觉反馈 + VR 身体动作控制 + 机器人执行 = 基础闭环遥操作
```

16. 阶段顺序总览

```
阶段 0：原始 HTS 手部追踪验证
阶段 1：脚部和腿部关节扩展
阶段 2：全身 84 个关节输出
阶段 3：Python 实时全身可视化
阶段 4：全身数据录制
阶段 5：记录与可视化同步进行
阶段 6：录制数据离线回放
阶段 7：提取机器人控制关键点
阶段 8：人体到机器人坐标转换
阶段 9：接入人形机器人模拟器
阶段 10：离线人体动作驱动机器人
阶段 11：实时 VR 到机器人模拟器控制
阶段 12：机器人视觉回传到 VR
阶段 13：闭环 VR 遥操作
```

建议执行顺序：

```
先确认数据可靠。
再提取控制关键点。
再做坐标和尺度标定。
再用离线数据驱动模拟器。
再改成实时控制。
最后加入机器人视觉回传。
```

17. 风险和注意事项

- Meta Body Tracking 是模型推断结果，不是高精度动作捕捉。
- 腿部和脚部数据可能在部分姿态下不稳定。
- 机器人不能直接使用人体关节点，需要 IK 和约束。
- 用户身体比例和机器人比例不同，需要做尺度归一化。
- 实时控制真实机器人前，需要先在模拟器中验证。
- 真实机器人控制需要急停、限位和速度限制。
- VR 视觉回传存在延迟，需要单独测量和优化。
- 网络抖动会影响实时控制，需要加入断连保护。
- 坐标系错误会导致机器人动作方向错误，需要单独做标定和可视化验证。
- IK 不可达目标需要裁剪，不能直接让机器人追踪任意人体位置。